Lab 0. Setting up the Environment for the Training

The Oracle Cloud Infrastructure (OCI) Compute lets you provision and manage compute hosts, known as instances. You can launch instances as needed to meet your compute and application requirements. In this lab, you will create a virtual machine/compute instance.

In this lab, you will:

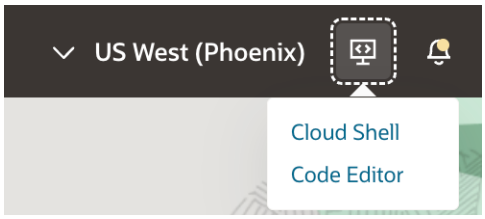
- Launch Cloud Shell and Generate SSH keys
- Create a VCN with public Subnet
- Create a Compute Instance
- Update the firewall on the instance
- Start a Jupyterlab server
- Access the Instance with VNC Client
- Deploy Kubernetes Dashboard on the Kubernetes Cluster

Prerequisites

- The Oracle University lab team has set up all the IAM policies required for you to complete this lab.
- Select the region available in the tenancy allotted to you. In this lab, **US West (Phoenix)** is considered as your region.

Sequence 1 Launch Cloud Shell and Generate SSH Key pair.

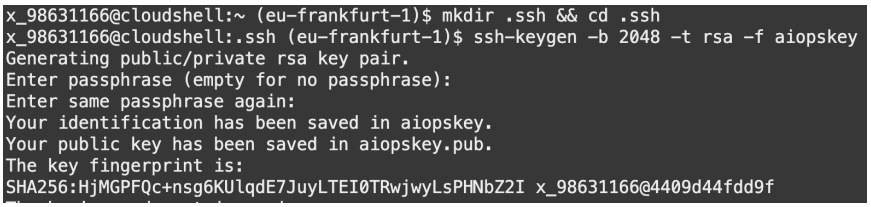
1. Sign in to your OCI Console with given credentials. When you login for the first time, you must change the password. ***Save new password in a notepad file***.
2. In the Console ribbon at the top of the screen, click the Region icon to expand the menu. Ensure that you are in the correct region, **US West (Phoenix)**.
3. Click the **Cloud Shell** icon next to the Region in the Console ribbon.



4. After the Cloud Shell has started, run the following commands (Type **N** to Skip Tutorial):

```
mkdir .ssh && cd .ssh
ssh-keygen -b 2048 -t rsa -f aiopskey
```

After entering the command above, press **Enter** twice for no passphrase.



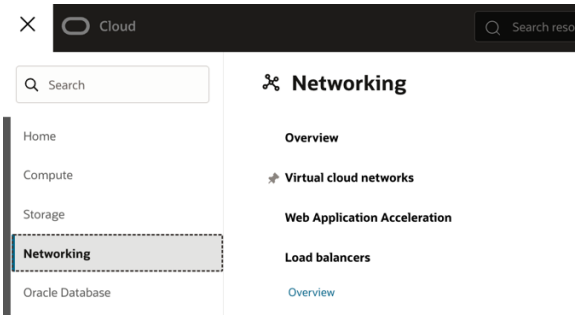
5. To list the contents of the public key, run the following command:

```
cat ~/.ssh/aiopskey.pub
```

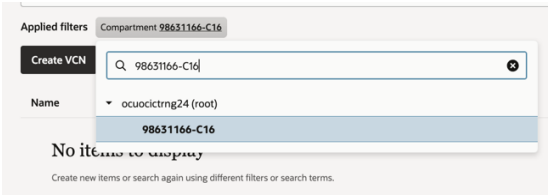
6. Keep the browser tab open to copy the contents of the public key (entire output) as you will require this in a subsequent step.

Sequence 2. Create a VCN with Public Subnet

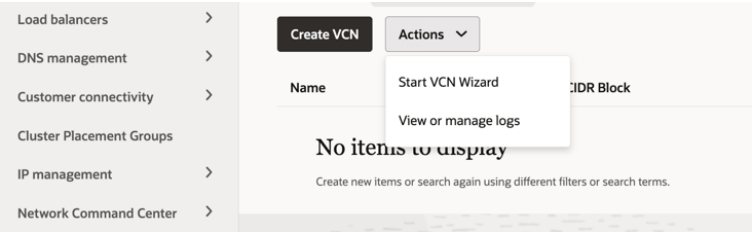
1. Launch Web Console in another Tab. From the **Main Menu**, under **Networking**, click **Virtual Cloud Network**.



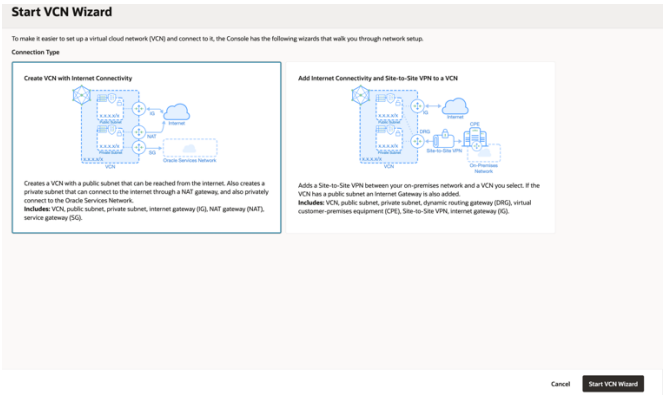
2. Select your compartment by clicking on link for "**Applied filters**". Compartment name is of the form "<Event ID>-C<User No>".



3. Click on "**Actions**" drop down and select "**Start VCN Wizard**", as given in screen shot below:



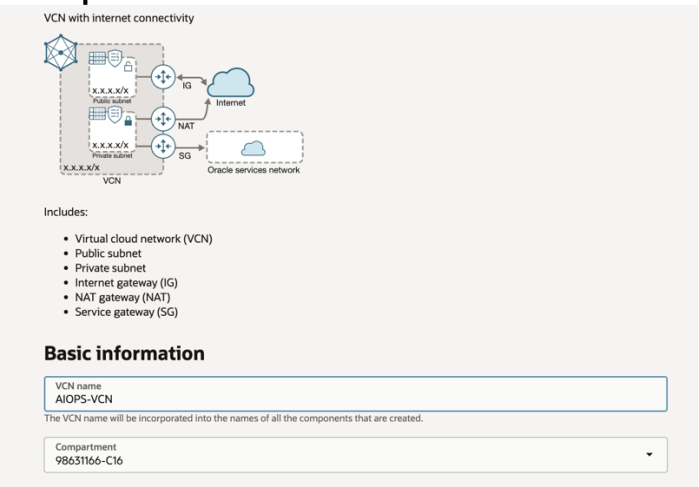
4. From Connectivity, keep "**Create VCN with Internet Connectivity**" default selection and click on select "**Start VCN Wizard**" from the bottom.



5. Fill in the details :

Basic information

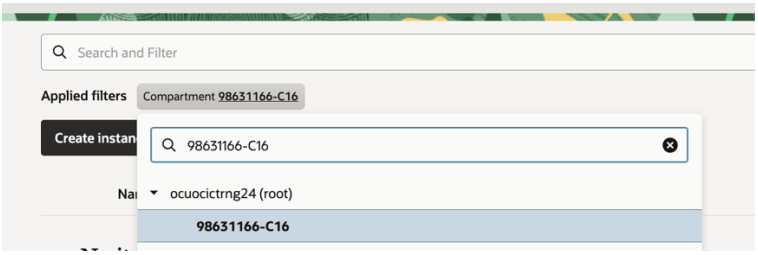
- VCN Name: **AIOPS-VCN**
- Compartment : Ensure your compartment is selected.
- Other Settings : **Keep defaults**



6. Click on "**Next**" at the bottom right.
7. Click on **Create** at the bottom right. VCN gets created.
8. Click in "**View VCN**" to verify.

Sequence 3. Create a Compute Instance

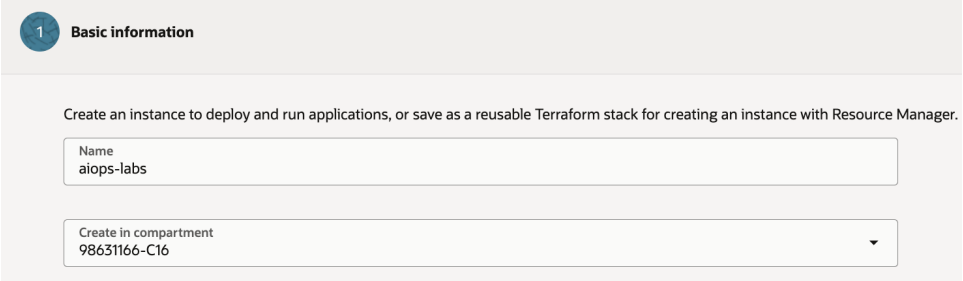
1. From the OCI Console **Main Menu**, under **Compute**, click **Instances**.
2. Ensure your compartment is selected. If not, select it by clicking on link for "**Applied filters**". Compartment name is of the form "<Event ID>-C<User No>" as given in screen shot below.



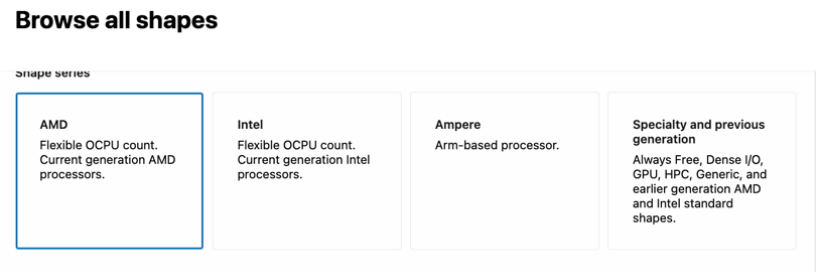
3. Click **Create instance**.
4. In the **Create compute instance** dialog box, populate the following information:

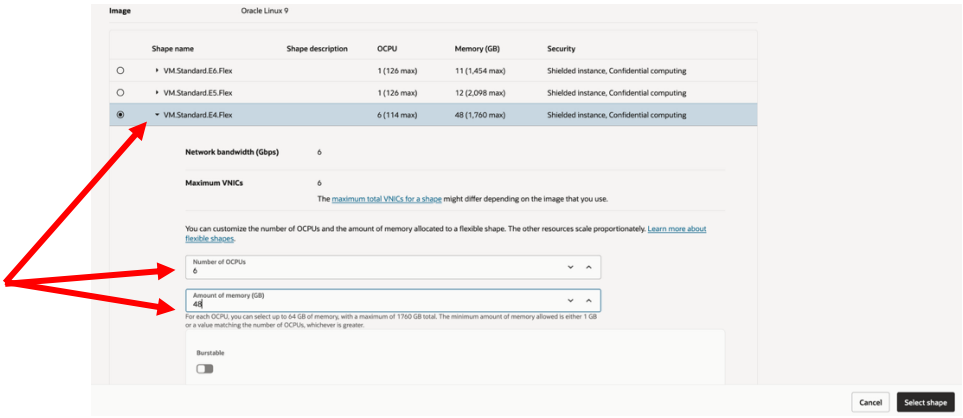
(1) Basic Information

- **Name**: aiops-labs
- **Create in compartment**: Select <*your compartment*>
- **Placement (Availability domain)**: AD 1



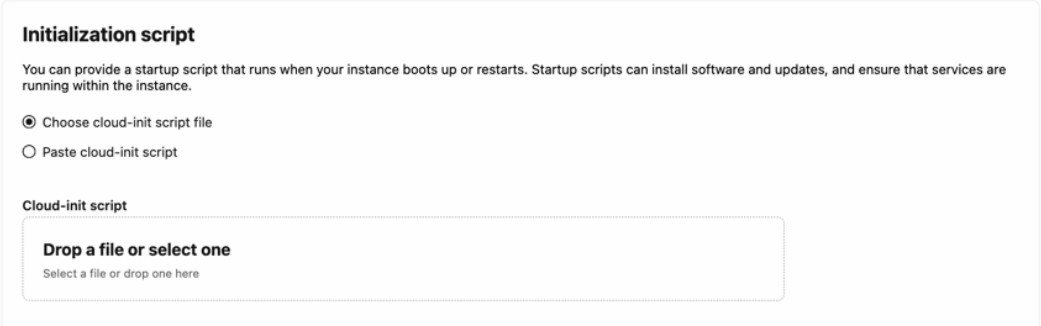
- **Image**: Oracle Linux 9 (Default)
- **Shape**: Select ***VM.Standard.E4.Flex*** (Under AMD)
  - **Number of OCPUs**: 6,
  - **Amount of memory (GB):** 48





- **Show advanced option**: Choose **cloud-init** script file

  Browse and upload the file **init-aiops.sh** provided by instructor.



- Click **Next** Button from bottom right

(2) Security. Click **Next** Button from bottom right again to skip Security

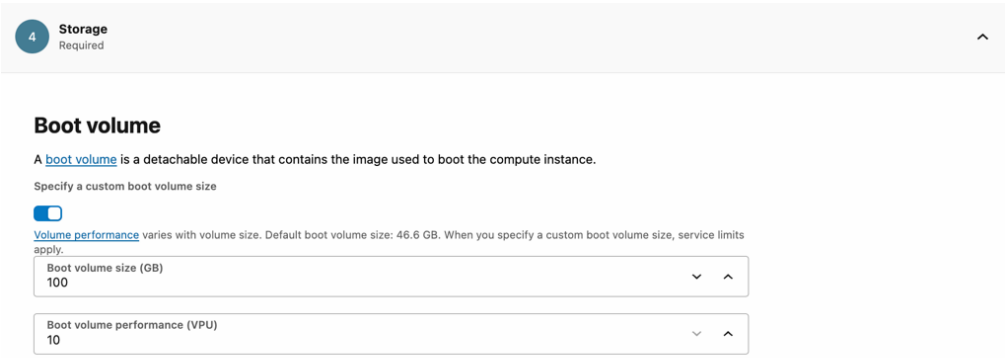
(3) Networking and SSH Keys:

VNIC Name - (Leave blank)

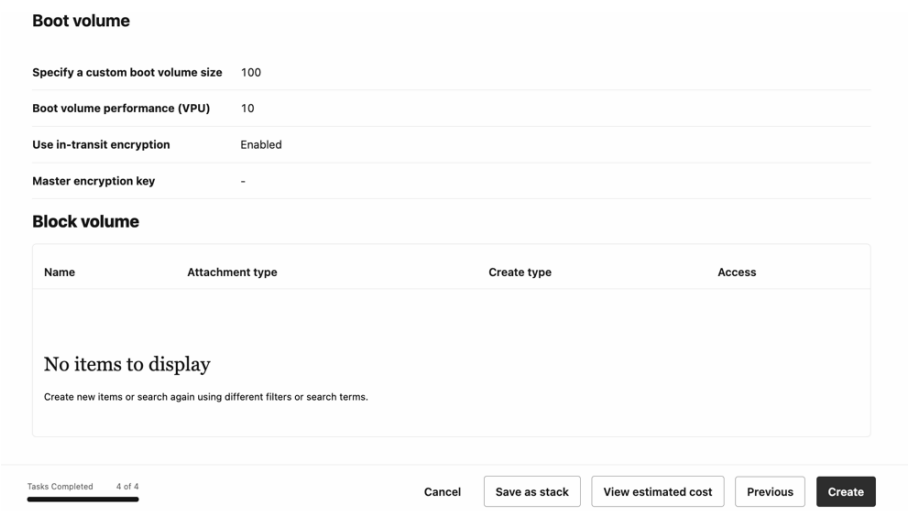
- **Primary network**: Select existing virtual cloud network
  - **Virtual cloud network: AIOPS-VCN**
- **Subnet:** Select existing subnet.
  - **Subnet:** public **subnet-AIOPS-VCN**
- **Private IPv4 address:** Automatically assign private IPv4 address
- **Automatically assign public IPv4 address :** Ensure it is selected.
- **Add SSH keys**: Paste public keys.
- **SSH Keys**: <*public key*> (Paste the public key you copied in **Step 5 of Sequence 1**)

(4) Storage :

- **Boot volume**: Specify a custom boot volume size
  - Boot volume size (GB) : 100 GB (Default is 50 GB)

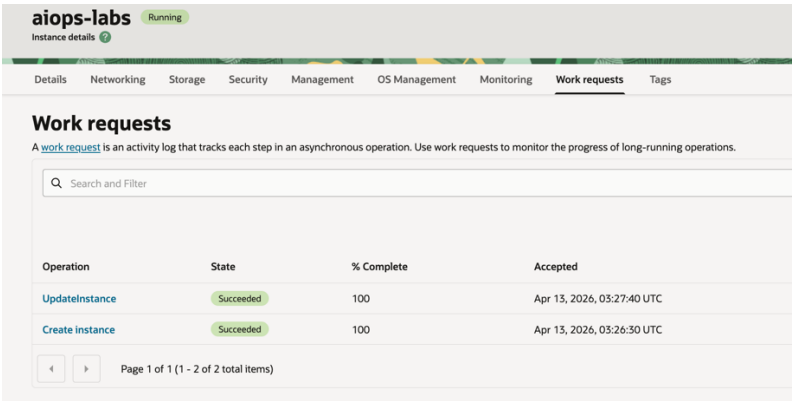


- Click **Next** Button from bottom right

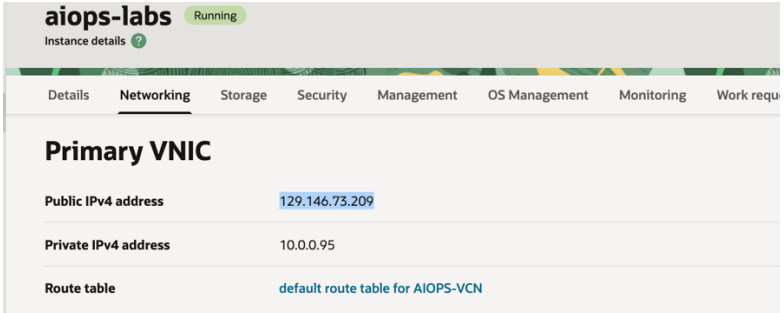


- Review and Click **Create**.

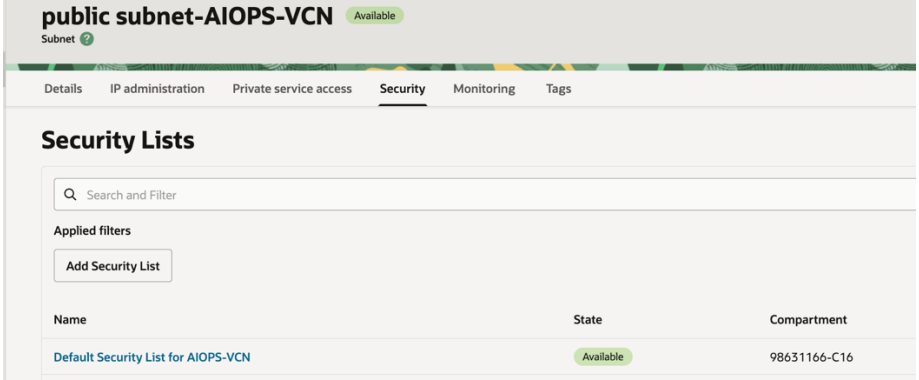
You will see that the Instance is created successfully, and the state is **Running**.



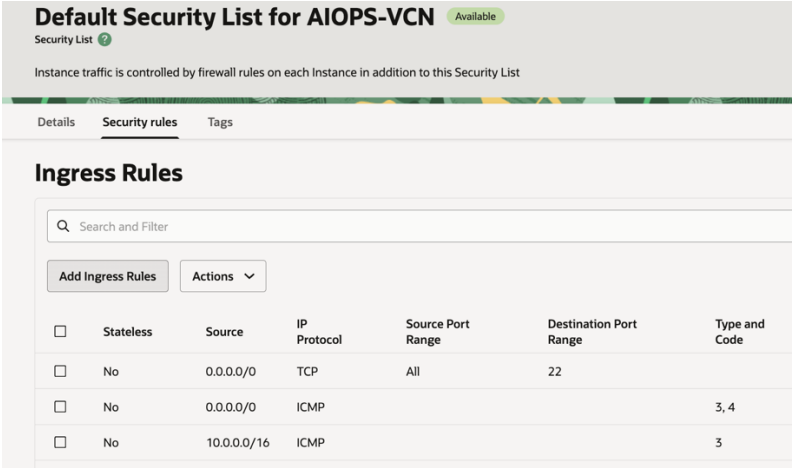
5. From **Networking** Tab, copy the Public IP and paste it in the Notepad.



6. Under the same tab, click on the **public subnet-AIOPS-VCN.** This will take you to **Subnet Details** page.
7. Under **Security Tab,** click on the **Default Security List for AIOPS-VCN.**



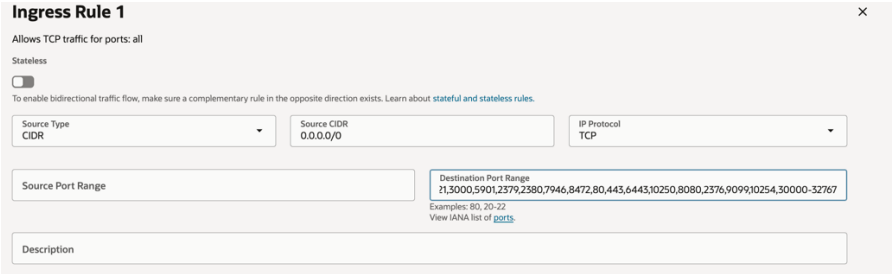
8. Click on **Security Rules** Tab and **Add Ingress Rules** Button



9. In the **Add Ingress Rules** dialog box, populate the following information:
   a. **Source CIDR**: 0.0.0.0/0.
   b. IP Protocol: TCP

   Destination Port Range:

   8888,8501,8000,1521,3000,5901,2379,2380,7946,8472,80,443,6443,10250,8080,2376,9099,10254,30000-32767



   - Do not select the **Stateless** check box.
   - Do not leave any space after the comma(,) while specifying the ports.

10. Click **Add Ingress Rule**.
11. Click the **Cloud Shell** icon next to the Region at the top of the screen and run the following command using SSH to connect to your instance:

```
cd
ssh -i ~/.ssh/aiopskey  opc@<public-ip>
```

Replace `<public-ip>` *with actual Public IP you copied in step 5 above.*

- Enter **yes** in response to - `Are you sure you want to continue connecting (yes/no)?`

```
x_98631166@cloudshell:.ssh (eu-frankfurt-1)$ ssh -i ~/.ssh/aiopskey  opc@129.146.73.209
The authenticity of host '129.146.73.209 (129.146.73.209)' can't be established.
ECDSA key fingerprint is SHA256:v/nvW/LBv9xqqb199Hz+lb097ObFPFfngb8WQo40YWY.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '129.146.73.209' (ECDSA) to the list of known hosts.
[opc@aiops-labs ~]$
```

You are now connected to the instance **aiops-labs**.

12. The bash script is running in the background. You can use the following command in the Cloud Shell to see the progress of script execution.

```
sudo tail -f /var/log/cloud-init-output.log
```

It will take about 35 Min for the execution of the Cloud Init script to complete. This command is used to **continuously monitor the last part of the cloud-init log file**. Once the entire script is executed you should see

```
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/opc/labs/.git/
Initialized empty Git repository in /home/opc/labs/.git/
From https://github.com/ou-developers/ou-generativeai-pro
 * branch            main       -> FETCH_HEAD
From https://github.com/ou-developers/ou-generativeai-pro
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
 * [new branch]      main       -> origin/main
Files successfully downloaded to /home/opc/labs
Files successfully downloaded to /home/opc/labs
===== Cloud-Init Script Completed Successfully =====
===== Cloud-Init Script Completed Successfully =====
Cloud-init v. 23.4-7.0.1.el8_10.8 running 'modules:final' at Sun, 09 Feb 2025 05:05:25 +0000. Up 20.41 seconds.
Cloud-init v. 23.4-7.0.1.el8_10.8 finished at Sun, 09 Feb 2025 05:11:42 +0000. Datasource DataSourceOracle.  Up 397.97 seconds
^C
[opc@gen-ai-labs ~]$
```

13. After the Execution of the Script is complete it will reboot the Instance.

Sequence 4. Configure Kubernetes (K3s) on the server.

1. It will take approx. 35 minutes for the script execution to complete. Once it is completed the instance will restart. Connect using up arrow key with same command and edit deployment metrics-server to allow self signed certificates

```
kubectl edit deployment metrics-server -n kube-system
```

a. Add Near Line No 46 (Use Space for indentation. Not Tab)

```
- --kubelet-insecure-tls
```

b. Near Line No 91

```
hostNetwork: true
```

```
    metadata:
      creationTimestamp: null
      labels:
        k8s-app: metrics-server
      name: metrics-server
    spec:
      containers:
      - args:
        - --cert-dir=/tmp
        - --secure-port=4443
        - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
        - --kubelet-use-node-status-port
        - --metric-resolution=15s
        - --kubelet-insecure-tls
        image: rancher/mirrored-metrics-server:v0.5.2
-- INSERT --
```

```
        terminationMessagePath: /dev/terminatio
        terminationMessagePolicy: File
        volumeMounts:
        - mountPath: /tmp
          name: tmp-dir
      dnsPolicy: ClusterFirst
      hostNetwork: true
      priorityClassName: system-node-critical
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      serviceAccount: metrics-server
      serviceAccountName: metrics-server
      terminationGracePeriodSeconds: 30
      tolerations:
```

2. Verify the Installation

```
kubectl get pods -o wide --all-namespaces
```

```
[root@server ~]# kubectl get pods -o wide --all-namespaces
NAMESPACE     NAME                                      READY   STATUS      RESTARTS
kube-system   coredns-d76bd69b-s2mtz                    1/1     Running     0
kube-system   helm-install-traefik-bjkpc                0/1     Completed   1
kube-system   helm-install-traefik-crd-tjvjh            0/1     Completed   0
kube-system   local-path-provisioner-6c79684f77-mc6dj   1/1     Running     0
kube-system   metrics-server-7cd5fcb6b7-d7q2b           1/1     Running     0
kube-system   svclb-traefik-5c921b88-tvjcb              2/2     Running     0
kube-system   traefik-df4ff85d6-fh8n4                   0/1     Running     0
[root@server ~]#
```

Sequence 5. Deploy Kubernetes Dashboard.

1. Download the compatible manifest for Kubernetes Dashboard. (2.6.1). If the download fails, visit the specified page on GitHub page and copy paste the content.

   ```
   kubectl apply -f \
   https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
   ```

2. Create two files; `admin-user.yaml` and `admin-user-role.yaml` with given contents.

   ```
   vi admin-user.yaml
   ```

   <Paste the Contents from the file provided>

   ```
   vi admin-user-role.yaml
   ```

   <Paste the Contents from the file provided>

3. Create Admin user, Role and Role Binding by applying these files (Contents shared)

```
kubectl apply -f admin-user.yaml
kubectl apply -f admin-user-role.yaml
```

4. Edit the Dashboard Deployment to allow access through node port by changing the "Type" property from "ClusterIP" to "NodePort" towards the end.

```
kubectl patch svc kubernetes-dashboard \
 -n kubernetes-dashboard \
 -p '{"spec": {"type": "NodePort"}}'
```

5. Now get the port to access the dashboard with

```
kubectl get svc kubernetes-dashboard -n kubernetes-dashboard
```

```
[root@server k3s-dashboard]# kubectl get svc kubernetes-dashboard -n kubernetes-dashboard
NAME                   TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)         AGE
kubernetes-dashboard   NodePort   10.43.238.95   <none>        443:32607/TCP   4m29s
[root@server k3s-dashboard]# kubectl -n kubernetes-dashboard describe secret admin-user-token
Name:         admin-user-token-gj6s7
```

In my Kubernetes cluster, I received a port number 32607 as shown in the above output. In your case, it might be different.

6. Fetch the Token to access the dashboard

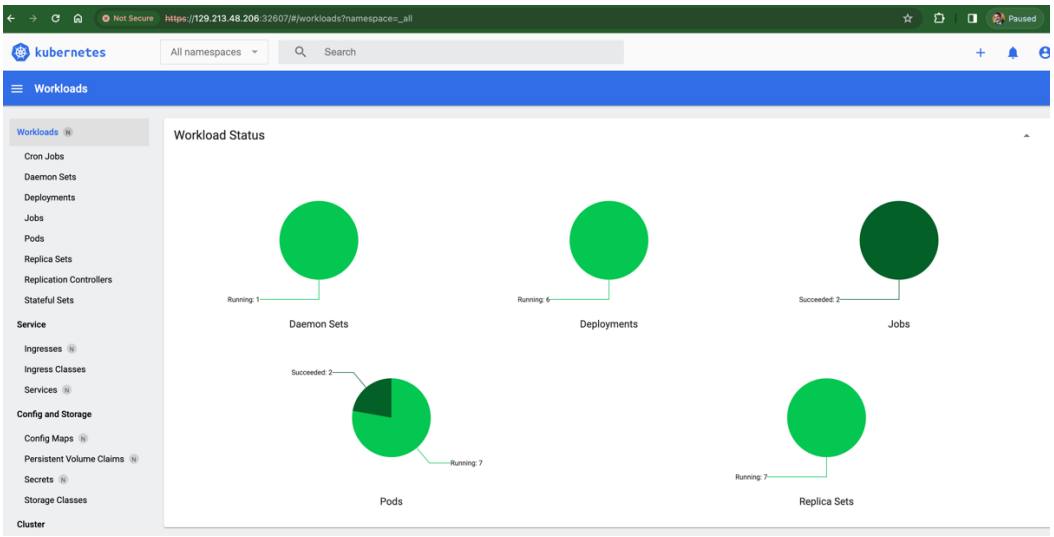
```
kubectl -n kubernetes-dashboard \
describe secret admin-user-token | grep "^token"
```


```
token:      eyJhbGciOiJSUzI1NiIsImtpZCI6Ind6YTJQRFU5bGFNTTRPb3ZISlRmazYyb1ZNRjlVakJNbUV1MzdkNVVOZUUifQ.eyJpc3
ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlcm5ldGVzLWRhc2hib2FyZCIsImt1YmVybmV0ZXMuaW8vc2Vydml
yLXRva2VuLWdqNnM3Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImFkbWluLXVzZXIiLCJr
Y2UtYWNjb3VudC51aWQiOiJiMTkzNDg5NS01YWE3LTRkMTYtOGZhMi1mYTY3NzI3ZTIzYzYiLCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY291b
iJ9.sB8zWGg389dMdADR6QJId623CPy5kiZhcHdsbZJQmXcFOKqFFlIyU917NtAsyNEDuqxF7pENLNrUASOYAKFKQ5pyeB3srP0LEaFAiUKBe
IUAdUtKelvam7yILoWwMaiXfpW63b5bsA1y5-kSSFx4tYN2eqEBAdmfyMKZa4dx4HD8sGzGycIjiy7HEjdLjEa8bD4z1FuLVxjkjfTkQi92vg
dsEUVZ5TdrwJLPySIBgsELYQna45ne6PvsCRux9w
```


7. Launch the Firefox browser from with instance to access the dashboard with https

https://<Public IP>:<port number>
Accept the Warning and Proceed. Enter the Token as password.

8. You will be on the Dashboard ag given below. Explore the Dashboard

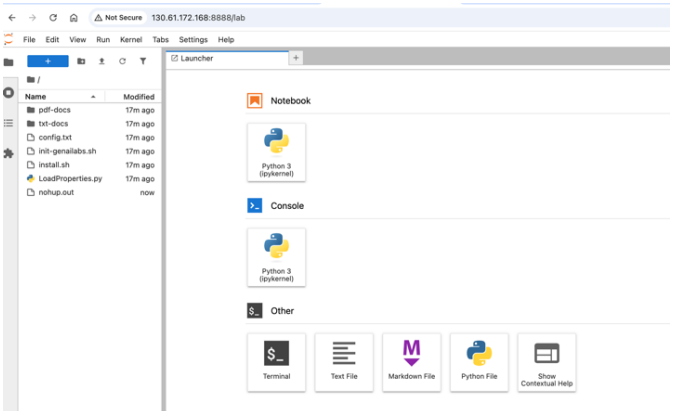


9. Congratulations! You have successfully completed this lab.

Sequence 6. Access the JupyterLab Server

1. Open a browser on your computer and type the following

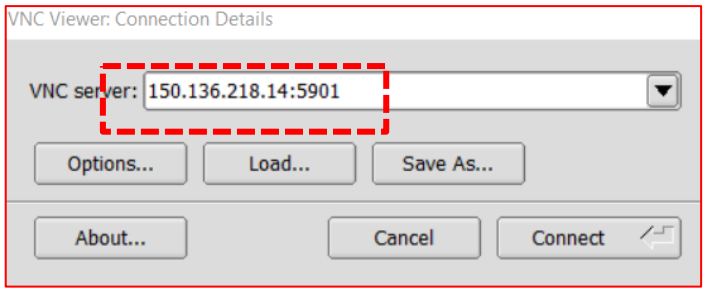
   `http://<public-ip-address>:8888`



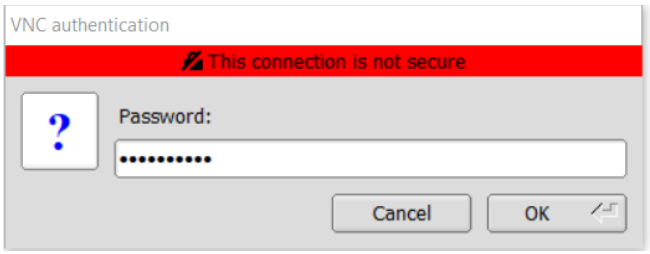
   Use `'Sangwan^123'` *as Password to login*

Sequence 7. Connecting to Desktop via VNC Client

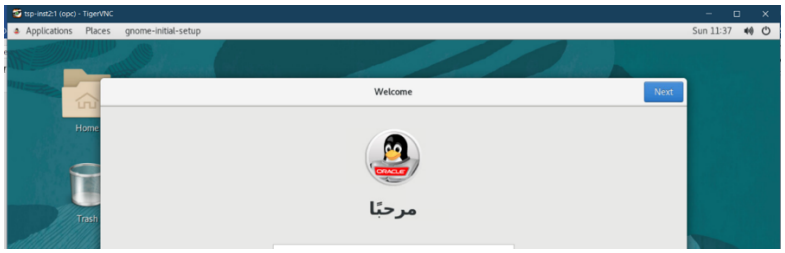
1. Install, if required, **TigerVNC Viewer** on your Mac/Windows Machine. For Mac, you can download it from this Link.
2. Start TigerVNC Viewer and Enter IP address with port number as given in the screen shot



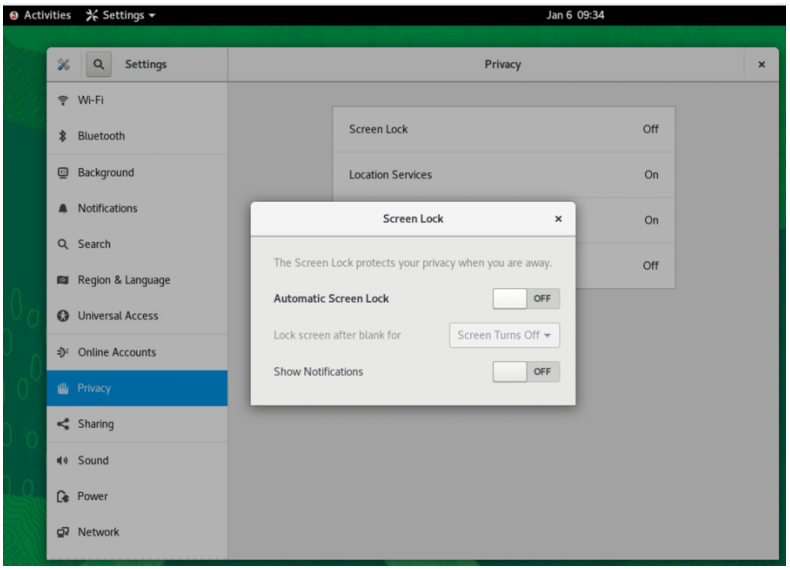
3. After the VNC connection has been established a warning saying that the connection is not secure. Also, it will ask the password. Use "**Oracle@123**" as given in **Step 15** of **Sequence 1** and Click OK.



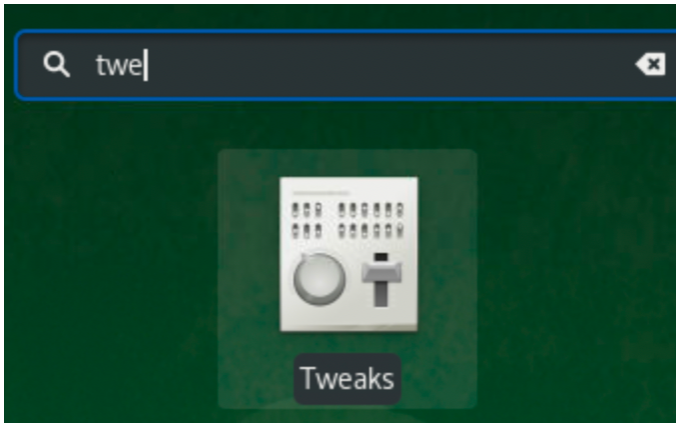
4. It will start your Session as given in the screen shot.

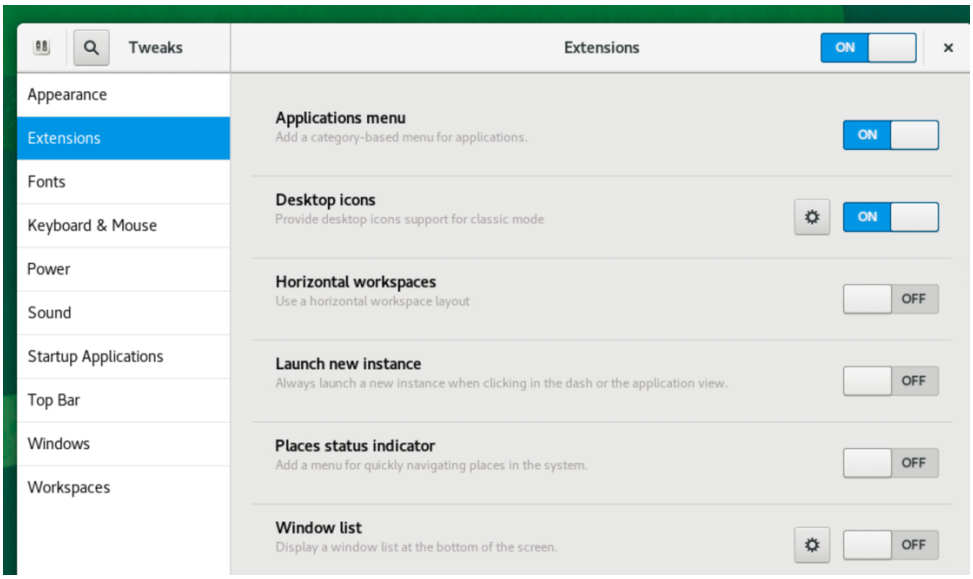


5. Disable screen lock by visiting **Activities - > Show Applications -> Settings - > Privacy**



6. Enable Desktop Icons and Application Menu, if you want **Activities - > Show Applications -> Tweaks**





7. **Congratulations! You have successfully completed this lab.**